

# Day 1 — JavaScript Senior Interview Cheat Sheet

Easy definitions • Examples • Senior interview points • Memory tricks

## Execution Context

**Easy Definition:** Environment JavaScript creates to run code. Think **Memory + Execution**.

**Example:**

```
function add(a,b){ return a+b; }  
add(2,3);
```

**Senior Point:** Function Execution Context is created when a function is invoked and pushed onto the call stack.

**Memory Trick:** **Memory + Execution** — “Pehle jagah banao, phir code chalo.”

## Call Stack

**Easy Definition:** Tracks currently executing functions using **LIFO**.

**Example:**

```
function one(){ two(); }  
function two(){ console.log('two'); }  
one();
```

**Senior Point:** two() is pushed after one() and therefore popped first.

**Memory Trick:** **Last in, first out.**

## Scope & Scope Chain

**Easy Definition:** Scope decides where a variable can be accessed. JavaScript uses **lexical scope**.

**Example:**

```
let x=10;  
function outer(){ let x=20; function inner(){ let x=30; console.log(x); } inner(); }
```

**Senior Point:** Lookup follows the lexical parent chain when a local variable is absent.

**Memory Trick:** **Lexical = location where code is written.**

## Closures ■

**Easy Definition:** A function can remember variables from its lexical scope even after the outer function finishes.

**Example:**

```
function counter(){ let count=0; return ()=>++count; }  
const next=counter();  
next(); //1  
next(); //2
```

**Senior Point:** Useful for private state, factories, callbacks and debounce.

**Memory Trick:** **Function + remembered lexical environment = Closure.**

## Closure + var / let

**Easy Definition:** var is function-scoped; let is block-scoped.

**Example:**

```
for(var i=0;i<3;i++) setTimeout(()=>console.log(i),0); //3 3 3  
for(let i=0;i<3;i++) setTimeout(()=>console.log(i),0); //0 1 2
```

**Senior Point:** var callbacks share one binding; let creates a separate iteration binding.

**Memory Trick:** **var = shared box; let = separate box.**

## Closure & Memory

**Easy Definition:** Closure itself is not automatically a memory leak. It can keep reachable objects alive.

**Example:**

```
function create(){ const big=new Array(1000000).fill('x'); return ()=>big.length; }
```

**Senior Point:** Use heap snapshots/retaining paths; clean listeners, timers, subscriptions and unnecessary caches.

**Memory Trick:** Leak = unintended reachability.

## this

**Easy Definition:** For normal functions, this is determined mainly by the call-site. Arrow functions capture this lexically.

**Example:**

```
const user={name:'Raj', regular(){console.log(this.name)}, arrow:()=>console.log(this.name)};
user.regular(); //Raj
user.arrow(); //usually undefined
```

**Senior Point:** Do not assume an arrow function gets this from the surrounding object.

**Memory Trick:** Regular: Who called me? Arrow: I borrow this.

## call / apply / bind

**Easy Definition:** call invokes now with individual args; apply now with array-like args; bind returns a function for later.

**Example:**

```
function greet(city){console.log(this.name,city)}
greet.call({name:'Raj'}, 'Delhi');
greet.apply({name:'Raj'}, ['Noida']);
greet.bind({name:'Raj'})('Pune');
```

**Senior Point:** call/apply execute immediately; bind does not.

**Memory Trick:** CALL=now, APPLY=now+array, BIND=later.

## Event Loop

**Easy Definition:** Sync code runs on the stack. Promise reactions are microtasks; timers are task/timer callbacks.

**Example:**

```
console.log('A');
setTimeout(()=>console.log('B'),0);
Promise.resolve().then(()=>console.log('C'));
console.log('D');
// A D C B
```

**Senior Point:** Microtasks are processed before moving to the next timer/task.

**Memory Trick:** Sync → microtasks → timers/tasks.

## Node process.nextTick

**Easy Definition:** Node gives process.nextTick special priority before normal Promise microtasks in typical execution.

**Example:**

```
console.log('1');
Promise.resolve().then(()=>console.log('2'));
process.nextTick(()=>console.log('3'));
console.log('4');
// 1 4 3 2
```

**Senior Point:** Node event loop also has phases such as timers, poll and check.

**Memory Trick:** nextTick = very high priority.

## Promise

**Easy Definition:** Represents an eventual async result. States: Pending → Fulfilled or Rejected.

**Example:**

```
const p=new Promise(resolve=>resolve('Success'));
p.then(v=>console.log(v)).catch(console.error);
```

**Senior Point:** Once settled, it does not return to pending.

**Memory Trick:** Pending → Success OR Failure.

## Promise Chaining

**Easy Definition:** Each then returns a new Promise. Return a value → next then gets it. Return a Promise → chain continues after that Promise settles.

**Example:**

```
getUser().then(user=>{ return getOrders(user.id); }).then(orders=>console.log(orders));
```

**Senior Point:** Always return the async Promise when the next step depends on it.

**Memory Trick:** return value → next gets value; return Promise → next waits.

## async / await

**Easy Definition:** Readable Promise-based syntax. await pauses that async function's continuation; it does not block the JS thread.

**Example:**

```
async function getData(){ const user=await getUser(); return await getOrders(user.id); }
```

**Senior Point:** Use try/catch for errors and sequential await when there is dependency.

**Memory Trick:** await pauses the function, not the runtime.

## Sequential vs Concurrent

**Easy Definition:** Independent operations can be started together to reduce latency.

**Example:**

```
const user=await getUser();
const [orders,products]=await Promise.all([getOrders(user.id),getProducts()]);
```

**Senior Point:** If durations are 1s, 2s, 1s: roughly  $1 + \max(2,1) = 3s$ .

**Memory Trick:** First ask: dependent or independent?

## Promise.all

**Easy Definition:** Fulfills when all fulfill; rejects if any rejects. Results preserve input order.

**Example:**

```
const [users,products]=await Promise.all([getUsers(),getProducts()]);
```

**Senior Point:** Does not automatically cancel other operations after one rejects.

**Memory Trick:** ALL = all required.

## Promise.allSettled

**Easy Definition:** Waits for every Promise and reports fulfilled/rejected status.

**Example:**

```
const results=await Promise.allSettled([getOrders(),getProducts()]);
```

**Senior Point:** Good when partial success is acceptable.

**Memory Trick:** Everyone reports.

## Promise.race / any

**Easy Definition:** race returns the first settled result; any returns the first fulfilled result.

**Example:**

```
await Promise.race([api1(),api2()]);
await Promise.any([server1(),server2()]);
```

**Senior Point:** race can fail first; any ignores rejections until all fail.

**Memory Trick:** RACE=first finish; ANY=first success.

## Error Handling

**Easy Definition:** Catch, retry, fallback, transform, log, or propagate deliberately.

**Example:**

```
async function load(){ try{return await fetchData();} catch(e){ console.error(e); throw e; } }
```

**Senior Point:** Avoid silently swallowing errors; choose behavior based on business requirements.

**Memory Trick:** **Catch** → **Decide** → **Handle/Propagate**.

## 30-Second Revision Map

| Concept                  | Remember                                                     |
|--------------------------|--------------------------------------------------------------|
| Execution Context        | Memory + Execution — “Pehle jagah banao, phir code chalao.”  |
| Call Stack               | Last in, first out.                                          |
| Scope & Scope Chain      | Lexical = location where code is written.                    |
| Closures                 | Function + remembered lexical environment = Closure.         |
| Closure + var / let      | var = shared box; let = separate box.                        |
| Closure & Memory         | Leak = unintended reachability.                              |
| this                     | Regular: Who called me? Arrow: I borrow this.                |
| call / apply / bind      | CALL=now, APPLY=now+array, BIND=later.                       |
| Event Loop               | Sync → microtasks → timers/tasks.                            |
| Node process.nextTick    | nextTick = very high priority.                               |
| Promise                  | Pending → Success OR Failure.                                |
| Promise Chaining         | return value → next gets value; return Promise → next waits. |
| async / await            | await pauses the function, not the runtime.                  |
| Sequential vs Concurrent | First ask: dependent or independent?                         |
| Promise.all              | ALL = all required.                                          |
| Promise.allSettled       | Everyone reports.                                            |
| Promise.race / any       | RACE=first finish; ANY=first success.                        |
| Error Handling           | Catch → Decide → Handle/Propagate.                           |

**Senior interview formula:** Definition → Internal working → Example → Why/when → Trade-off → Production scenario.